Structures de données pour l'unification en logique du premier ordre

Rapport en vue de la validation de l'UE Initiation à la recherche

Année universitaire 2025-2026

Etudiants :
Ayath ABOGOUNRIN
Guillaume POTEL
Matheo VIGNERES

Encadrants :
Julie CAILLER
Marie DUFLOT-KREMER
Engel LEFAUCHEUX

DÉCHARGE DE RESPONSABILITÉS

« L'Université de Lorraine n'entend donner ni approbation ni improbation aux opinions émises dans ce rapport, ces opinions devant être considérées comme propres à leur auteur. »

REMERCIEMENTS

Nous tenons à remercier toutes les personnes qui ont contribué au bon déroulement de notre projet et qui nous ont aidé dans la rédaction de ce rapport.

Tout d'abord, nous voulons remercier chaleureusement l'ensemble de nos encadrants pour leur bienveillance, leur pédagogie, leur temps et surtout leur bonne humeur tout au long du projet. Nous remercions donc Madame Julie Cailler, pour son soutien émotionnel sans faille, spécifiquement face aux Discrimination Trees, Madame Marie Duflot-Kremer pour ses remarques toujours pertinentes et son choix judicieux de date d'anniversaire, nous ayant permis de profiter d'un délicieux goûter, et enfin Monsieur Engel Lefaucheux pour ses précieux conseils, notamment pour les benchmarks finaux, et ses propositions hebdomadaires de chocolats qui, nous tenons à le préciser au lecteur, ont toujours été refusées.

Aussi, nous souhaitons remercier Monsieur Didier Galmiche, notre professeur de Logique cette année, sans qui notre choix de sujet ne se serait sûrement jamais porté sur cette thématique passionnante, challegeante et pleine de (bons comme mauvais) rebondissements.

Enfin, nous remercions toutes les personnes nous ayant conseillé et relu lors de la rédaction de ce rapport.

Table des matières

Introduction **1**

1 Généralités **2**
1.1 Logique des propositions . . . 2
1.2 Logique du premier ordre . . . 3
1.3 Recherche de preuve . . . 5
1.3.1 Forme clausale et réfutation . . . 5
1.3.2 Règle de résolution . . . 6
1.4 Unification . . . 6

2 Structures de données et algorithmes **9**
2.1 Structures de données représentant un terme . . . 9
2.1.1 Problématique . . . 9
2.1.2 Intuition . . . 9
2.1.3 Implémentation . . . 9
2.2 Structures de données représentant un ensemble de prédicats . . . 10
2.2.1 Liste . . . 11
2.2.2 Ensemble . . . 11
2.2.3 Dictionnaire . . . 12
2.3 Algorithmes d'unification . . . 13
2.3.1 Robinson . . . 13
2.3.2 Martelli-Montanari . . . 16
2.4 Discrimination Tree . . . 18
2.4.1 Présentation . . . 18
2.4.2 La structure arborescente . . . 18
2.4.3 L'algorithme d'unification . . . 20
2.4.4 Complexité . . . 22

3 Expérimentations **25**
3.1 Organisation et implémentation . . . 25
3.2 Protocole de benchmarks . . . 26
3.3 Hypothèses . . . 28
3.4 Analyse des résultats . . . 29

Conclusion **30**

Introduction

En informatique, le choix des **structures de données** est une décision fondamentale : elle conditionne directement l'efficacité des algorithmes qui les manipulent. Ce principe, universel en programmation, prend une dimension particulière dans le domaine de la **logique**, où la représentation des objets manipulés doit non seulement refléter fidèlement une syntaxe formelle stricte, mais aussi permettre un traitement automatique efficace. C'est précisément ce défi qui est au cœur de ce rapport.

La **logique du premier ordre** est un langage formel permettant d'exprimer des propriétés et des relations sur des objets. Elle repose sur des expressions symboliques appelées **termes**, construits à partir de **variables**, de **constantes** et de **symboles de fonction**. Ces termes constituent la brique de base des **formules logiques**, et leur manipulation efficace est un enjeu central dans de nombreux systèmes automatiques. Parmi les opérations fondamentales sur ces termes, l'**unification** occupe une place de choix : elle consiste à déterminer s'il existe une **substitution** de variables permettant de rendre deux termes identiques. Cette opération intervient notamment dans les **moteurs d'inférence**, les **systèmes de réécriture de termes** et les **prouveurs de théorèmes automatiques**.

Dans ce contexte, une question naturelle se pose : quelles sont les **structures de données** les plus adaptées pour représenter et manipuler efficacement des termes dans le cadre de l'unification ? Le choix d'une structure impactant directement l'efficacité des algorithmes qui l'exploitent, nous nous interrogerons également sur les **algorithmes d'unification** les mieux adaptés à chaque représentation. C'est autour de cette double problématique, structures et algorithmes, que s'articule ce travail de recherche.

Nous commencerons par établir les **généralités** nécessaires à la compréhension du sujet : logique des propositions, logique du premier ordre, preuve automatisée et unification. Nous présenterons ensuite les **structures de données** retenues pour représenter les termes et les organiser en base, ainsi que les **algorithmes d'unification** étudiés, en particulier le **discrimination tree**. Enfin, nous exposerons les **résultats expérimentaux** obtenus lors de nos benchmarks, avant de conclure sur les enseignements tirés de cette étude. L'ensemble du code source est disponible dans le dépôt git du projet : `https://github.com/Metheor31Game/M1-recherche`.

1. Généralités

D'après nos connaissances, l'UE de L3 intitulée Logique des propositions - Logique du premier ordre*, de Jean Lieber [1] et l'UE de M1 intitulée* Logiques et Modèles de Calcul*, de Didier Galmiche.*

Fil conducteur.

Tout au long de ce chapitre, nous allons construire progressivement les outils nécessaires pour répondre à une question concrète : comment une machine peut-elle *démontrer* automatiquement que « Socrate est mortel » d'après les hypothèses « Tout homme est mortel. » et « Socrate est un homme » ? Ce raisonnement, connu sous le nom de *syllogisme*, peut sembler trivial au lecteur mais requiert, pour être mécanisé, des outils formels précis. Nous introduirons d'abord brièvement la logique des propositions, sur laquelle s'appuie la logique du premier ordre, avant de présenter cette dernière puis l'unification, les deux notions centrales de ce rapport.

1.1. Logique des propositions

Définition.

La *logique des propositions* (ou logique propositionnelle) repose sur une idée simple : on représente des *faits élémentaires*, par exemple « Socrate est un homme », auxquels on attribue une *valeur de vérité* ($vrai$ ou $faux$). Ces *faits élémentaires* sont appelés *variables propositionnelles*, notées conventionnellement p, q, r, etc. Ces *variables* peuvent ensuite être combinés à l'aide de *connecteurs logiques* pour former des *formules*.

Voici une définition formelle d'une *formule* en *logique propositionnelle* :

$$\langle \text{formule} \rangle ::= \langle \text{variable propositionnelle} \rangle \mid \neg(\langle \text{formule} \rangle) \mid (\langle \text{formule} \rangle\ \langle \text{connecteur binaire} \rangle\ \langle \text{formule} \rangle)$$

$$\langle \text{connecteur binaire} \rangle ::= \wedge \mid \vee \mid \rightarrow \mid \leftrightarrow$$

Remarque. *Les connecteurs ont les lectures suivantes :*

$\neg$	$\wedge$	$\vee$	$\rightarrow$	$\leftrightarrow$
« non »	*« et »*	*« ou »*	*« implique »*	*« équivaut à »*

Exemple : traduction du syllogisme.

On peut encoder notre syllogisme en posant :

p	« Socrate est un homme »
q	« Tout homme est mortel »
r	« Socrate est mortel »

et former la *formule* $(p \wedge q) \rightarrow r$, qui se lit « si Socrate est un homme et tout homme est mortel, alors Socrate est mortel ».

Limites.

La logique propositionnelle ne traite que d'un individu à la fois : elle est insuffisante pour exprimer des propriétés générales portant sur des ensembles d'objets ou des relations entre eux.

Revenons à notre syllogisme, on *encode en dur* chaque fait : la proposition q est un bloc opaque ; on ne peut pas en extraire la règle générale « pour tout individu x, si x est un homme, alors x est mortel ». Si l'on voulait raisonner sur d'autres individus, e.g. Platon ou Aristote, il faudrait introduire autant de nouvelles *variables propositionnelles*, sans jamais capturer la généralité du raisonnement.

C'est précisément cette limite que lève la *logique du premier ordre*, présentée dans la section suivante, en introduisant des *variables* sur lesquelles on peut quantifier.

1.2. Logique du premier ordre

Pour pouvoir étendre le champs de couverture de la *logique propositionnelle*, on a défini la *logique du premier ordre* (ou logique des prédicats).

Cette logique permet la représentation de propriétés portant sur des ensembles d'objets ou des relations. Pour les décrire au sein de *formules*, on se donne les éléments suivants :

— Ensemble de *variables* : x, y, z, etc. Elles représentent des individus *indéterminés* ;

— Ensemble de symboles de *fonctions* : f, g, h, etc. Elles permettent de construire de nouveaux objets à partir d'objets existants, e.g. $pere(socrate)$ désigne le père de Socrate ;

— Ensemble de symboles de *prédicats* : P, Q, R, etc. Ils expriment des propriétés ou des relations entre objets, e.g. $Homme(socrate)$ exprime que Socrate est un homme. Par convention, leurs noms commencent par une majuscule.

On associe à ces fonctions et prédicats une *arité* (i.e. le nombre d'arguments qu'ils prennent). Par exemple, *pere* est d'arité 1 car il prend un seul argument, tandis qu'un prédicat comme $EstParentDe(x, y)$ est d'arité 2 car il met en relation deux individus. Parmi les symboles de fonctions, ceux d'arité 0 sont appelés *constantes* :

— Ensemble de symboles de constantes : a, b, c, etc. Elles représentent des individus précis et fixes, e.g. *socrate*.

> **Remarque.** *Le choix de ces différents symboles est purement arbitraire : on pourrait nommer les variables a, b ou c. Dans un souci d'harmonisation et de standardisation, on utilisera néanmoins les symboles les plus courants dans la littérature.*

Pour définir la structure d'une *formule* en *logique du premier ordre*, il nous faut définir la notion d'*atome* qui s'appuie elle même sur la notion de *terme*.

Terme.

Un *terme* est une expression désignant un *objet*. Il peut être une *variable*, une *constante* ou un *symbole de fonction* appliqué à d'autres *termes*.

Formellement, on défini un *terme* par la grammaire suivante :

$$\langle \text{terme} \rangle ::= \langle \text{variable} \rangle \mid \langle \text{constante} \rangle \mid \langle \text{symbole de fonction} \rangle (\underbrace{\langle \text{terme} \rangle, \ldots, \langle \text{terme} \rangle}_{n \text{ termes}})$$

où n correspond à l'arité du symbole de fonction.

Par exemple, avec *socrate* comme constante, x comme variable et *pere* comme symbole de fonction d'arité 1, on peut construire les *termes* :

$$socrate \qquad x \qquad pere(socrate) \qquad pere(x)$$

> **Remarque.** *Un terme* n'est pas *une formule, il désigne un objet, il n'exprime pas une propriété.*

Atome.

Là où un *terme* désigne un *objet*, un *atome* exprime une *propriété* sur des objets. Il est formé d'un *symbole de prédicat* appliqué à des termes. Formellement :

$$\langle \text{atome} \rangle ::= \langle \text{prédicat} \rangle (\underbrace{\langle \text{terme} \rangle, \ldots, \langle \text{terme} \rangle}_{n \text{ termes}})$$

où n correspond à l'arité du prédicat.

Par exemple, avec *Homme* et *Mortel* comme prédicats d'arité 1, *socrate* comme constante et x comme variable :

$$Homme(socrate) \qquad Mortel(socrate) \qquad Mortel(x)$$

sont des atomes. $Homme(socrate)$ exprime que Socrate est un homme, $Mortel(x)$ exprime que l'individu désigné par x est mortel.

> **Remarque.** *Un littéral (ou des littéraux) est un prédicat auquel on a ajouté un signe.*
> *Exemple : $P(f(x))$ et $\neg P(f(x))$*

Formule.

Les *atomes* peuvent être combinés à l'aide des connecteurs logiques vus en *logique propositionnelle*, et enrichis par deux *quantificateurs* :

- le *quantificateur universel* $\forall$ (« pour tout ») : $\forall x\, \varphi$ signifie que φ est vraie quelle que soit la valeur attribuée à x ;
- le *quantificateur existentiel* $\exists$ (« il existe ») : $\exists x\, \varphi$ signifie qu'il existe au moins une valeur de x pour laquelle φ est vraie.

Une *formule* de la *logique du premier ordre* est alors définie par :

$$\begin{aligned}\langle \text{formule} \rangle ::= {} & \langle \text{atome} \rangle \mid \neg(\langle \text{formule} \rangle) \mid \\ & (\langle \text{formule} \rangle\ \langle \text{connecteur binaire} \rangle\ \langle \text{formule} \rangle) \mid \langle \text{quantificateur} \rangle\ \langle \text{variable} \rangle\ (\langle \text{formule} \rangle)\end{aligned}$$

Par exemple, « pour tout individu x, si x est un homme alors x est mortel » s'écrit :

$$\forall x \left(Homme(x) \rightarrow Mortel(x) \right)$$

Ce que la *logique propositionnelle* ne pouvait pas exprimer, i.e. la généralité du raisonnement sur tout individu x, est ici capturé par une unique *formule quantifiée*.

1.3. Recherche de preuve

Définition.

De manière informelle, une *preuve* est une suite finie d'étapes de *dérivation* : chaque étape produit une nouvelle formule à partir de formules déjà établies (les hypothèses ou les formules dérivées précédemment), en appliquant une règle d'inférence autorisée. Une telle suite est dite valide si elle part des *prémisses* et aboutit à la *conclusion* souhaitée en ne franchissant que des étapes justifiées. Dans le cadre de la *résolution* présentée ci-après, une preuve prend la forme d'une suite de *résolvantes* dérivées deux à deux, jusqu'à l'obtention de la *clause vide* $\square$, témoin d'une contradiction.

Nous disposons maintenant des outils pour formaliser proprement notre syllogisme.

Introduisons la *signature* suivante :

— Constante : *socrate*

— Symboles de prédicats : *Homme* d'arité 1, *Mortel* d'arité 1

Les deux prémisses s'écrivent alors :

$\forall x \, \big(Homme(x) \to Mortel(x)\big)$	tout homme est mortel
$Homme(socrate)$	Socrate est un homme

Et l'on cherche à démontrer d'après les deux prémisses :

$Mortel(socrate)$	Socrate est mortel

Remarque. *En logique propositionnelle, il aurait fallu créer une proposition distincte pour chaque individu. Ici, une seule formule quantifiée $\forall x$ capture la règle générale, applicable à n'importe quelle constante.*

1.3.1. Forme clausale et réfutation

La mise sous *forme clausale* consiste à transformer un ensemble de formules *clauses*, i.e. des disjonctions de littéraux, qui leur est logiquement équivalent.

Pour notre syllogisme, la formule $\forall x \; (Homme(x) \to Mortel(x))$ est d'abord réécrite sous la forme $\forall x \; (\neg Homme(x) \vee Mortel(x))$, en utilisant l'équivalence logique entre $A \to B$ et $\neg A \vee B$. Les quantificateurs universels étant implicites, on obtient ainsi la clause $\neg Homme(x) \vee Mortel(x)$.

La technique de *réfutation* consiste, plutôt que de démontrer directement une conclusion, à ajouter sa *négation* aux prémisses et à chercher une *contradiction*. Si une contradiction est trouvée, cela prouve que la conclusion est bien vraie.

En ajoutant la négation de $Mortel(socrate)$ à nos prémisses et en les mettant sous forme clausale, on obtient trois clauses :

$$
\begin{aligned}
C_1 &= \neg Homme(x) \vee Mortel(x) \\
C_2 &= Homme(socrate) \\
C_3 &= \neg Mortel(socrate)
\end{aligned}
$$

Il nous reste à dériver une contradiction à partir de ces trois clauses, ce que réalise la *règle de résolution*, présentée ci-après.

Remarque. *Il existe un théorème, que l'on utilise ici, qui nous dit que toute formule de la logique du premier ordre peut être mise sous forme clausale. Cette transformation, bien que non triviale dans le cas général, est toujours possible et constitue une étape préalable indispensable à l'application de la règle de résolution.*

1.3.2. Règle de résolution

La *règle de résolution* permet de dériver une nouvelle clause à partir de deux clauses existantes. Si une clause contient un littéral L et une autre contient sa négation $\neg L$, on peut en dériver une nouvelle clause, appelée *résolvante*, contenant tous les autres littéraux des deux clauses :

$$\frac{L \vee C \qquad \neg L \vee D}{C \vee D}$$

Cependant, les littéraux ne sont pas toujours syntaxiquement identiques : dans notre exemple, C_1 contient $Homme(x)$ et C_2 contient $Homme(socrate)$. Ces deux littéraux partagent le même symbole de prédicat, mais leurs arguments diffèrent : l'un est une variable x, l'autre une constante *socrate*. Pour pouvoir appliquer la règle de résolution, il faut d'abord les rendre identiques en *substituant* x par *socrate*. C'est précisément ce que réalise l'*unification*, que nous définirons dans la section suivante.

1.4. Unification

Substitution.

Une *substitution* σ est une fonction qui associe à certaines variables des termes. Par exemple, $\sigma = \{X \mapsto b,\ Y \mapsto f(a)\}$ associe la variable X au terme b et la variable Y au terme $f(a)$. Appliquer σ à un terme t, noté $\sigma(t)$, consiste à remplacer simultanément chaque occurrence des variables concernées par leur terme associé.

Par exemple, si $t = g(X, Y)$, alors :

$$\sigma(t) = \sigma(g(X, Y)) = g(b, f(a))$$

Unification.

L'*unification* consiste à déterminer s'il existe une substitution σ, appelée *unificateur*, telle que $\sigma(t_1) = \sigma(t_2)$ pour deux termes t_1 et t_2 donnés. Si une telle substitution existe, on dit que t_1 et t_2 sont *unifiables*.

Parmi tous les unificateurs possibles, on cherche le plus général, noté *MGU* (de l'anglais *Most General Unifier*), c'est-à-dire celui qui impose le moins de contraintes sur les variables. Il est dit « le plus général » car tout autre unificateur peut s'obtenir en composant le MGU avec une substitution supplémentaire.

Remarque. *L'unification n'est pas toujours possible. Par exemple, les termes $f(a)$ et $g(a)$ ne sont pas unifiables car leurs symboles de tête sont différents. De même, une variable x ne peut pas être unifiée avec un terme qui la contient déjà, comme $f(x)$, car cela produirait un terme de taille infinie.*

Exemple.

Considérons les deux termes suivants :

$$t_1 = f(X, Y) \qquad \text{et} \qquad t_2 = f(a, Z)$$

On cherche une substitution σ telle que $\sigma(t_1) = \sigma(t_2)$. La substitution

$$\sigma = \{X \mapsto a,\ Y \mapsto Z\}$$

unifie ces deux termes :

$$\sigma(t_1) = f(a, Z) = \sigma(t_2)$$

Cette substitution est le MGU, car elle réalise uniquement les substitutions strictement nécessaires. D'autres substitutions permettent également l'unification, par exemple :

$$\sigma_1 = \{X \mapsto a,\ Y \mapsto b,\ Z \mapsto b\}$$

ce qui donne :

$$\sigma_1(t_1) = f(a, b) = \sigma_1(t_2)$$

Ou encore :

$$\sigma_2 = \{X \mapsto a,\ Y \mapsto g(c),\ Z \mapsto g(c)\}$$

pour laquelle :

$$\sigma_2(t_1) = f(a, g(c)) = \sigma_2(t_2)$$

Ces substitutions sont plus spécifiques que le MGU, car elles s'obtiennent en composant σ avec une substitution supplémentaire appliquée à Z. En revanche, la substitution $\sigma_3 = \{X \mapsto b,\ Y \mapsto Z\}$ n'est pas un unificateur, car :

$$\sigma_3(t_1) = f(b, Z) \neq f(a, Z) = \sigma_3(t_2)$$

Ainsi, toute substitution unificatrice s'obtient à partir du MGU

$$\{X \mapsto a,\ Y \mapsto Z\}$$

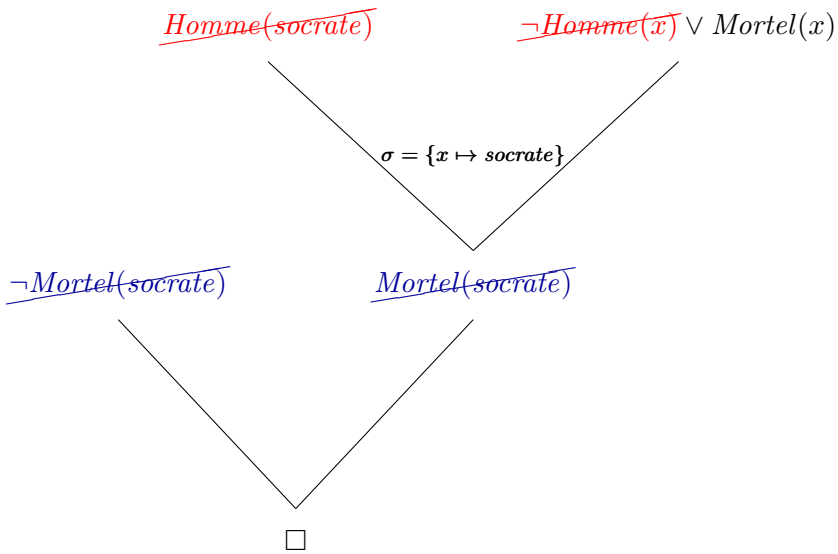
par spécialisation.

Clôture du fil conducteur.

Revenons à nos trois clauses obtenues à la section précédente :

$$\begin{aligned} C_1 &= \neg Homme(x) \vee Mortel(x) \\ C_2 &= Homme(socrate) \\ C_3 &= \neg Mortel(socrate) \end{aligned}$$

Pour appliquer la règle de résolution sur C_1 et C_2, il faut unifier $Homme(x)$ et $Homme(socrate)$. La substitution $\sigma = \{x \mapsto socrate\}$ est leur MGU. On peut alors dériver la contradiction en deux étapes :



littéraux résolus à l'étape 1 : $\sigma_1 = \{x \mapsto socrate\}$
littéraux résolus à l'étape 2 : $\sigma_2 = \emptyset$

— **Étape 1 :** On résout C_1 et C_2. Les littéraux $Homme(x)$ et $Homme(socrate)$ sont unifiés par $\sigma = \{x \mapsto socrate\}$, ce qui produit la résolvante $Mortel(socrate)$.
— **Étape 2 :** On résout $Mortel(socrate)$ et $C_3 = \neg Mortel(socrate)$. Les deux littéraux sont identiques, aucune autre substitution n'est nécessaire. On obtient la *clause vide* $\square$, témoin de la contradiction.

La négation de $Mortel(socrate)$ est donc incompatible avec nos prémisses, ce qui prouve que $Mortel(socrate)$ est bien une conséquence du syllogisme. Cet exemple illustre le rôle central de l'unification dans la démonstration automatique : sans elle, la règle de résolution ne pourrait pas instancier les variables et serait donc inutilisable en pratique. La question de l'*efficacité* de cette unification, en fonction des structures de données et des algorithmes utilisés, est précisément l'objet du reste de ce rapport.

Dans notre exemple, nous n'avons raisonné que sur trois clauses. Dans un système réel de *recherche de preuve*, cet ensemble peut contenir des milliers de *littéraux* issus de la mise sous *forme clausale* de la base de connaissances initiale. On appelle *base de connaissances* cet ensemble de clauses sur lequel le moteur d'inférence, i.e. le composant logiciel chargé d'appliquer les règles de déduction, opère : c'est à partir de cette base que la *règle de résolution* est appliquée de manière répétée, en cherchant à chaque étape des paries de littéraux unifiables.

L'efficacité de ce processus dépend donc directement de la façon dont cette base est *organisée* et *stockée* : parcourir l'intégralité de la base pour trouver un candidat à l'unification à chaque étape de résolution représente un coût prohibitif sur de grands ensembles. La question centrale deviant alors : quelle *structure de données* adopter pour stocker la base de connaissance de façon à rendre l'*unification* la plus efficace possible ? C'est précisément l'objet de la partie suivante, qui procédera en deux temps : nous verrons d'abord comment représenter un *terme*, composante élémentaire de la base de connaissances, avant de nous intéresser aux structures permettant de stocker et d'interroger efficacement la base dans son ensemble. Notons que nous nous concentrons sur la représentation d'un *terme* plutôt que d'un *prédicat*, car un prédicat n'est autre qu'un *symbole de prédicat* appliqué à des *termes*, sa représentation découle donc naturellement de celle de ses composantes.

2. Structures de données et algorithmes

2.1. Structures de données représentant un terme

2.1.1. Problématique

La définition d'un terme en logique du premier ordre impose des contraintes naturelles sur sa représentation : un terme est soit une variable, soit une constante, soit une fonction appliquée à d'autres termes. Toute structure de données adéquate doit donc être capable d'exprimer cette hiérarchie, et de distinguer ces trois cas, car ils se comportent différemment lors de l'unification :

— une *variable* peut être substituée par n'importe quel terme ;

— une *constante* est un symbole fixe sans arguments ;

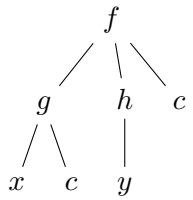
— une *fonction* est un symbole appliqué à un nombre précis de sous-termes, déterminé par son arité.

Notons que, formellement, une constante est une fonction d'arité 0. Nous choisissons néanmoins de les distinguer explicitement, cette séparation étant naturelle et utile en pratique lors de l'unification.

2.1.2. Intuition

La structure d'un terme est naturellement hiérarchique : une fonction est composée de sous-termes, qui peuvent eux-mêmes être des fonctions, et ainsi de suite. Cette imbrication définit la *profondeur* du terme. Les variables et les constantes, en revanche, sont toujours des *feuilles*, i.e. elles n'ont pas de sous-termes, leur profondeur vaut 0.

Considérons le terme $t = f(g(x, c), h(y), c)$, où f est une fonction d'arité 3, g une fonction d'arité 2, h une fonction d'arité 1, x, y des variables et c une constante. La structure hiérarchique de t se représente naturellement sous forme d'arbre :



On constate que t est de profondeur 2 : la racine f est au niveau 0, les termes $g(x, c)$ et $h(y)$ ainsi que la feuille c au niveau 1, et enfin x, c et y, des feuilles, au niveau 2. C'est cette structure hiérarchique qui nous amène naturellement à proposer les *arbres* comme structure de données pour représenter un terme.

2.1.3. Implémentation

Testons cette idée sur la base B définie comme :

$$B = \{a,\ X,\ Y,\ c,\ f(X, Y),\ g(Y),\ g(a),\ f(a, X)\}$$

Cette représentation capture bien la hiérarchie des termes. Cependant, elle présente deux limites importantes. D'une part, rien ne distingue la variable X de la constante a, ce sont deux feuilles *structurellement* identiques. D'autre part, rien n'indique qu'un nœud f doit avoir exactement n fils, l'arité de la fonction n'est pas encodée. Or ces deux informations sont essentielles lors de l'unification : une variable peut être substituée, une constante non ; et deux fonctions ne peuvent être unifiées que si elles ont la même arité.

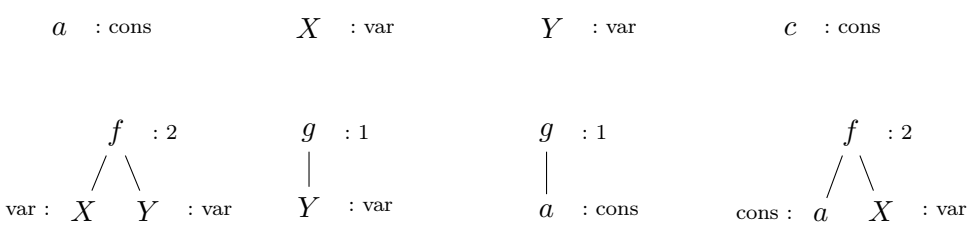
La solution naturelle est d'enrichir l'arbre en associant à chaque nœud une *étiquette* indiquant son type. On définit trois étiquettes possibles :

— var : le nœud est une variable, c'est nécessairement une feuille ;

— cons : le nœud est une constante, c'est nécessairement une feuille ;

— $n \in \mathbb{N}^*$: le nœud est une fonction d'arité n, il a exactement n fils.

Notre base B est maintenant représentée comme ceci :



Cette représentation enrichie permet désormais d'exprimer explicitement toutes les contraintes d'un terme : le *type* de chaque nœud est visible, l'arité des fonctions est encodée dans l'*étiquette*, et la structure hiérarchique est capturée par l'*arbre* lui-même.

> **Remarque.** *Les arbres étant un cas particulier de graphes, cette structure plus générale constitue une piste d'exploration naturelle. En représentant les termes par des graphes avec partage de sous-termes, la détection de cycles lors du parcours pourrait offrir une détection de l'*occur-check *quasi-native. Il s'agit là d'un embryon d'idée que nous n'avons pas eu l'occasion d'explorer dans le cadre de ce rapport.*

2.2. Structures de données représentant un ensemble de prédicats

Maintenant que la représentation d'un *terme* est acquise, nous cherchons à évaluer quelle structure de données est la plus *efficace* pour stocker et interroger la *base de connaissances* en vue de l'*unification*. Pour comparer ces structures, nous nous concentrerons sur trois opérations clés :

— l'**insertion** : ajouter un nouveau littéral à la base ;

— la **recherche** : retrouver les candidats potentiels à l'unification avec un littéral requête ;

— l'**unification** : appliquer l'algorithme d'unification sur les candidats retenus.

La *suppression* n'est pas une opération pertinente dans notre contexte : dans un processus de *résolution par réfutation*, la base de connaissances reste fixe, on y ajoute des clauses dérivées au fil des étapes, mais on ne supprime jamais de clauses existantes. Cette opération ne sera donc pas prise en compte dans notre analyse. Nous avons retenu trois structures principales pour nos comparaisons : les *listes*, les *ensembles* et les *dictionnaires*.

2.2.1. Liste

Principe général.

La liste est la structure séquentielle la plus élémentaire pour stocker notre base de prédicats. Elle conserve l'ordre d'insertion et autorise la présence de doublons.

Avantages.

— **Coût de construction quasi nul :** L'ajout d'un prédicat dans la base d'unification, i.e. l'ensemble des prédicats sur lequels on veut chercher une unification, se fait en $\mathcal{O}(1)$. Cela peut être avantageux dans les cas ou l'on souhaite unifier sans faire de prétraitement, sur une base construite à la volée et peu réutilisée.

— **Parcours brut rapide :** La nature séquentielle de la liste favorise une itération très efficace, le surcoût lié à l'accès en mémoire lors du passage d'un prédicat à l'autre est quasi inexistant.

Inconvénients.

— **Comparaison exhaustive obligatoire :** La contrepartie de cette simplicité est l'obligation d'une recherche aveugle. Pour unifier un candidat, il est impératif de le confronter un à un à tous les éléments de la liste (pour une liste de N prédicats, $\mathcal{O}(N)$ tentatives). Si l'on souhaite filtrer les prédicats incompatibles pour éviter ce balayage systématique, un prétraitement devient alors obligatoire.

— **Redondance des calculs :** La liste n'empêchant pas les doublons, si un même prédicat est présent plusieurs fois, l'algorithme d'unification sera exécuté de manière répétée et inutile.

Bilan pour l'unification.

La liste restant la structure séquentielle de référence par excellence, il était naturel de l'envisager en premier. Même en anticipant ses limites de performances sur de grands volumes de données, cette approche naïve reste toutefois intéressante à analyser.

2.2.2. Ensemble

Principe général.

Un ensemble (*Set*) est une collection non ordonnée qui garantit l'unicité de ses éléments. Dans notre contexte, il s'assure qu'aucun prédicat strictement identique n'est stocké deux fois dans la base.

Avantages.

- **Élimination native des doublons :** C'est son atout principal. En filtrant les prédicats redondants dès l'insertion, l'ensemble réduit mécaniquement la taille de la base. Cela garantit qu'un candidat ne sera jamais unifié deux fois avec le même prédicat, économisant potentiellement du temps.
- **Vérification d'existence immédiate :** Si l'on cherche une égalité stricte plutôt qu'une unification, l'ensemble permet de vérifier la présence d'un prédicat en $\mathcal{O}(1)$ moyen, contre $\mathcal{O}(N)$ pour une liste de taille N.

Inconvénients.

- **Coût d'insertion des termes :** Pour garantir l'unicité en $\mathcal{O}(1)$ lors de l'insertion, l'ensemble vérifie si le prédicat inséré n'est pas déjà présent, ce qui peut prendre un peu de temps.
- **Absence de filtrage pour l'unification :** Tout comme la liste, l'ensemble ne permet pas de pré-sélectionner les candidats à l'unification, e.g. des littéraux de même tête et signes opposés. Il faut toujours itérer sur l'intégralité de l'ensemble, menant aussi à $\mathcal{O}(N)$ tentatives pour un ensemble de N éléments.

Bilan pour l'unification.

L'ensemble améliore l'approche naïve en assainissant la base de données (pas de doublons). Le compromis se situe entre le temps supplémentaire passé lors de la phase d'insertion et le temps gagné lors de la phase de résolution (moins d'unifications inutiles). Toutefois, le problème du parcours exhaustif n'est toujours pas résolu.

2.2.3. Dictionnaire

Principe général.

Le dictionnaire permet de stocker des paires clé-valeur. Dans le cadre de l'unification, il introduit une première forme d'*indexation de prédicats* : au lieu de stocker les prédicats en vrac, on les classe dans le dictionnaire en utilisant une propriété structurelle comme clé (par exemple, le symbole de tête et son arité).

Avantages.

- **Filtrage des candidats :** C'est la première structure qui permet de réduire le nombre de tentatives d'unification. Si l'on cherche à unifier le prédicat $P(f(X,Y),Y)$, on interroge le dictionnaire avec la clé `'¬P/2'`. On récupère instantanément uniquement les prédicats complémentaires, i.e. même prédicat avec signe opposé, de la base.
- **Gain de performance en résolution :** En évitant les tentatives d'unification avec des prédicats structurellement incompatibles, i.e. symboles de prédicat différents, on passe d'une complexité de recherche de $\mathcal{O}(N)$ à $\mathcal{O}(K)$, où K est le nombre de prédicats partageant la même clé et N la taille du dictionnaire (avec $K \ll N$).

Inconvénients.

— **Complexité et coût mémoire :** Organiser les prédicats par clés demande davantage d'allocations mémoire (gestion de la table de hachage, listes sous-jacentes pour gérer les collisions ou les groupes de prédicats).

Bilan pour l'unification.

Le dictionnaire, bien qu'une structure plus complexe, marque la transition entre le stockage simple et l'indexation. En regroupant les prédicats par « famille » (symbole de tête/arité), il réduit considérablement le nombre d'appels à l'algorithme d'unification. Cette approche profite directement du fonctionnement intrinsèque des prédicats logiques pour optimiser la recherche, le temps investi lors de la mise en place de cette structure peut être bénéfique pour l'unification.

2.3. Algorithmes d'unification

2.3.1. Robinson

Origine et principe.

L'algorithme d'unification de Robinson est un processus algorithmique fondamental en logique mathématique et en informatique théorique. Publié en 1965 par John Alan Robinson, il constitue un pilier du principe de résolution, utilisé pour la démonstration automatique de théorèmes et la programmation logique.

Avant 1965, la démonstration automatique de théorèmes était entravée par l'explosion combinatoire des recherches de preuves. J. A. Robinson a révolutionné le domaine en publiant son article de référence : *A Machine-Oriented Logic Based on the Resolution Principle* [2]. Il y introduit l'unification comme un moyen de sélectionner intelligemment les substitutions nécessaires pour appliquer la règle de résolution, évitant de tester une infinité de combinaisons de manière aveugle.

L'objectif de l'algorithme de Robinson (ainsi que de tous ses successeurs) est de calculer le MGU entre deux littéraux, permettant de les rendre identiques via une substitution commune à tous les termes.

Fonctionnement de l'algorithme.

Le déroulement de cet algorithme s'effectue en plusieurs étapes pour unifier deux littéraux (voir Algorithme 1).

Pour trouver la substitution qui unifie deux termes, on utilise la fonction UnifierTermes, dont le fonctionnement est décrit dans l'algorithme 2 ci-après.

Algorithm 1 Unification de deux littéraux (Robinson)

```
Procédure UnifierLitteraux(L1, L2)
                                                  ▷ Étape 1 : Vérification des signes
    Si Signe(L1) == Signe(L2) alors
        Retourner Échec                           ▷ Pas de résolution possible si même signe
    Fin si
                                                  ▷ Étape 2 : Vérification du nom du prédicat
    Si Prédicat(L1) ≠ Prédicat(L2) alors
        Retourner Échec                           ▷ Les littéraux n'ont pas le même symbole de prédicat
    Fin si
                                                  ▷ Étape 3 : Vérification de l'arité
    Si Arité(L1) ≠ Arité(L2) alors
        Retourner Échec                           ▷ Nombre d'arguments différent
    Fin si
                                                  ▷ Étape 4 : Unification itérative des termes (enfants)
    Subst ← ∅                                     ▷ Initialisation de la substitution commune
    N ← Arité(L1)
    Pour i ← 1 à N faire
        T1 ← Enfant(L1, i)
        T2 ← Enfant(L2, i)
                                                  ▷ Unification en accumulant les substitutions précédentes
        Subst ← UnifierTermes(T1, T2, Subst)
        Si Subst == Échec alors
            Retourner Échec                       ▷ Dès qu'une paire échoue, tout le prédicat échoue
        Fin si
    Fin pour
    Retourner Subst                               ▷ Retourne la substitution commune (MGU)
Fin procédure
```

Algorithm 2 Unification de deux termes (Robinson)

```
Procédure UnifierTermes(T_1, T_2, Subst)
                                        ▷ Cas 1 : Les termes ont le même symbole (Nom et Type)
    Si Nom(T_1) == Nom(T_2) alors
        Si EstFonction(T_1) alors                    ▷ Cas trivial, pas besoin de substituer
                                        ▷ Unification récursive des arguments (enfants)
            Pour i ← 1 à Arité(T_1) faire
                S_i ← UnifierTermes(Enfant(T_1, i), Enfant(T_2, i))
                Si S_i == Échec alors
                    Retourner Échec
                Fin si
                Subst ← Subst ∪ S_i
            Fin pour
            Retourner Subst
        Sinon
            Retourner ∅          ▷ Constantes ou variables identiques : succès, aucune substitution
        Fin si
    Fin si
                                        ▷ Cas 2 : Le premier terme est une variable
    Si EstVariable(T_1) alors
        Si OccursCheck(T_1, T_2) alors
            Retourner Échec                          ▷ Cycle détecté (T_1 apparaît dans T_2)
        Fin si
        Retourner {T_1 ↦ T_2}
    Fin si
                                        ▷ Cas 3 : Le second terme est une variable
    Si EstVariable(T_2) alors
        Si OccursCheck(T_2, T_1) alors
            Retourner Échec                          ▷ Cycle détecté (T_2 apparaît dans T_1)
        Fin si
        Retourner {T_2 ↦ T_1}
    Fin si
                                        ▷ Cas 4 : Conflit (Clash)
    Retourner Échec                     ▷ Symboles différents ou arités incompatibles
Fin procédure
```

Remarque. *OccursCheck*(T_2, T_1) *est la fonction booléenne vérifiant qu'un terme* T_2 n'apparait pas *dans un terme* T_1*. C'est une vérification essentielle dans n'importe quel algorithme d'unification. Exemple :* *OccursCheck*$(f(X), g(f(X), Y))$ *renvoie donc true puisque le terme* $g(f(X), Y)$ *est composé de* $f(X)$*.*

2.3.2. Martelli-Montanari

Origine et principe.

L'algorithme Martelli Montanari (MM) a été publié en 1982 par Alberto Martelli et Ugo Montanari dans la revue ACM *Transactions on Programming Languages and Systems.* [3] Élément clé dans l'étude de l'unification du premier ordre, sa principale contribution est de modéliser l'unification comme un système de transformation, offrant ainsi un cadre formel qui permet de démontrer la correction de plusieurs algorithmes pratiques. Il est généralement considéré comme plus efficace que l'algorithme de Robinson dans le cas général [3]. Son objectif est de déterminer, pour un ensemble de termes du premier ordre, s'il existe une substitution qui les rend tous identiques deux à deux, et si tel est le cas, de produire l'unificateur le plus général sous une forme résolue.

Fonctionnement de l'algorithme.

L'algorithme consiste à considérer un système d'équation E de forme $E = \{s_1 \stackrel{?}{=} t_1, \ldots, s_n \stackrel{?}{=} t_n\}$ où les s_i et t_i sont des termes du premier ordre, et à itérer sur lui jusqu'à atteindre une forme résolue ou détecter un échec. Une équation $s_1 \stackrel{?}{=} t_1$ à la signification suivante : « Les terme s_1 et t_1 sont-ils unifiables ? ». Or, dans notre cas, les deux entrées sont des *prédicats* de la forme $P(S_1, \ldots, S_n)$ et $P(t_1, \ldots, t_n)$. Un prétraitement est donc nécessaire afin de pouvoir passer le système d'équation à l'algorithme de Martelli Montanari.

Algorithm 3 Création du système d'équations et unification de littéraux

```
Fonction LitListe(l1, list_litteraux, touteUnif = Vrai)
    resultat ← ∅
    Pour tout l2 ∈ list_litteraux faire        ▷ Étape 1 : Comparaison de l1 aux autres littéraux
                                                ▷ Étape 2 : Vérif. symbole, signe et arité
        Si l1.prdicat = l2.prdicat et l1.signe ≠ l2.signe et l1.arit = l2.arit alors
            S ← ∅                               ▷ Init. du système de termes (équations)
            Pour tout (t1, t2) ∈ (l1.enfants, l2.enfants) faire
                Ajouter l'équation t1 = t2 à S  ▷ Étape 3 : Ajout des paires d'enfants
            Fin pour
            σ ← MartelliMontanari(S)            ▷ Étape 4 : Résolution via Martelli-Montanari
            Si σ existe alors
                resultat[l2] ← σ
                Si non touteUnif alors
                    Retourner resultat          ▷ Retour dès le premier succès
                Fin si
            Sinon
                continuer                       ▷ Échec de l'unification, on continue
            Fin si
        Fin si
    Fin pour
    Retourner resultat
Fin fonction
```

Comment se passe l'unification au sein de Martelli Montanari ?
On applique les règles suivantes sur le système d'équation jusqu'a épuisement.

— **Delete :** consiste à supprimer une équation réflexive ($t \stackrel{?}{=} t$), c'est-à-dire où les termes de la partie gauche et de la partie droite sont les mêmes.

— **Décompose** $f(s_1, \ldots, s_k) \stackrel{?}{=} f(t_1, \ldots, t_k)$ **:** Lorsque deux termes partagent le même symbole de fonction f et la même arité, il suffit d'unifier leurs sous-termes. On remplace l'équation initiale $f(s_1, \ldots, s_k) \stackrel{?}{=} f(t_1, \ldots, t_k)$ par les équations $s_1 \stackrel{?}{=} t_1, \ldots, s_k \stackrel{?}{=} t_k$. Enfin on procède à son unification composante par composante.

— **Orient** $t \stackrel{?}{=} x$ **:** Par convention, les équations sont orientées de façon à toujours placer la variable à gauche. Si la partie gauche est un terme non-variable et la partie droite est une variable, on échange les deux membres.

— **Occur Check** $x \stackrel{?}{=} f(x)$ **:** consiste à vérifier si la variable x apparaît dans le terme t (sans en être égale), appliquer la substitution $x \to t$ produirait un terme infini, par exemple $x = f(f(f(\ldots)))$. Une telle solution n'existe pas, l'algorithme échoue puis retourne ECHEC.

— **Clash** $f(s_1, \ldots, s_n) \stackrel{?}{=} g(t_1, \ldots, t_m)$ **:** Si les deux termes ont des symboles de tête différents, aucune substitution ne pourra jamais les rendre égaux. L'équation n'a donc pas de solution, l'algorithme s'arrête et retourne ECHEC. Cette règle couvre également le fait de vouloir unifier une constante avec un terme composé ($a \stackrel{?}{=} f(x)$).

Bien que l'ordre d'application des règles ne change pas le résultat final, et de surplus l'algorithme ne prescrive pas d'ordre strict entre les règles, appliquer *Delete* et *Décompose* en priorité réduit plus rapidement la taille du système et améliore les performances en pratique. Une fois que toutes les règles de transformation ont été appliquées avec succès, le système d'équations est réduit à une forme résolue, c'est-à-dire un ensemble d'équations de la forme $\{x_1 \stackrel{?}{=} t_1, \ldots, x_k \stackrel{?}{=} t_k\}$, où chaque x_i est une variable distincte n'apparaissant dans aucun des termes t_j.

La substitution est alors construite en lisant directement ces liaisons : chaque variable x_i est associée au terme t_i auquel elle a été liée au cours du processus. Cette substitution $\sigma = \{x_1 \mapsto t_1, \ldots, x_k \mapsto t_k\}$ constitue le MGU du système initial.

La complexité de l'algorithme.

La complexité de notre implémentation est $O(n^2 \cdot |t|)$, où n désigne le nombre d'équations du système et $|t|$ la taille maximale d'un terme. Cette complexité est supérieure à la complexité théorique optimale de MM, qui est quasi-linéaire $O(n \cdot \alpha(n))$ [3]. L'écart s'explique par trois facteurs : premièrement, la représentation des termes sous forme d'arbres récursifs implique que toute opération (*get_ variables*, *substitution*) nécessite un parcours complet de l'arbre en $O(|t|)$; deuxièmement, le recalcul systématique des variables à chaque itération sans mise en cache ; et troisièmement, la reconstruction explicite des termes lors des substitutions, au lieu d'une structure union-find qui permettrait des substitutions en temps quasi-constant $O(\alpha(n))$.

2.4. Discrimination Tree

D'après Term Indexing - *R. Sekar, I.V. Ramakrishnan et A. Voronkov [4],* An Algorithm for the Retrieval of Unifiers from Discrimination Trees - *H. De Nivelle [5] et* On the Evaluation of Indexing Techniques for Theorem Proving - *R. Nieuwenhuis, T. Hillenbrand, A. Riazanov et A. Voronkov [6].*

2.4.1. Présentation

Le *discrimination tree*, ou *arbre de discrimination* en français (bien qu'aucun ouvrage francophone n'y fasse explicitement référence) est une véritable chimère algorithmique : une *structure de données arborescente* conçue spécifiquement pour l'indexation de termes (ou pour notre cas de prédicats) et un *algorithme* efficace d'unification. C'est en raison de sa nature particulière que nous décidons de la présenter au lecteur dans cette section qui lui est complètement dédiée.

Le *discrimination tree* nous a été présenté par nos encadrants, sans qui cette structure serait vraisemblablement restée dans l'ombre de notre étude. En effet, cette structure ne figurait pas dans notre périmètre initial : c'est précisément grâce à leurs conseils avisés et à la documentation qu'ils nous ont fournie que nous avons pu en saisir le potentiel et décider de l'intégrer pleinement à notre sujet. Ce choix s'est imposé naturellement au regard de ses propriétés théoriques : le *discrimination tree* se révèle en effet plus puissant que les approches algorithmiques classiques pour l'unification, justifiant ainsi amplement son inclusion. Nous les remercions à nouveau chaleureusement pour cette piste décisive.

2.4.2. La structure arborescente

Un *discrimination tree* est construit en insérant les termes de la base sous forme aplatie (*flatten*) : chaque terme est d'abord linéarisé en une *p-string* par un parcours préfixe, transformant ainsi sa structure hiérarchique en une séquence de symboles. Par exemple, le terme $f(g(a,b),a)$ se linéarise en $f - g - a - b - a$. Ces *p-strings* sont ensuite insérées dans un *arbre*, de sorte que les préfixes communs à plusieurs termes soient *factorisés* au maximum en un unique chemin partagé, évitant toute redondance et constituant ainsi l'index. Enfin, chaque chemin se termine par une feuille contenant un ou plusieurs pointeurs vers les objets originaux (termes ou prédicats), permettant ainsi de récupérer instantanément les données correspondantes une fois le parcours de l'arbre terminé.

Il existe plusieurs variantes du *discrimination tree*, se distinguant notamment par la façon dont les variables sont traitées lors de la construction de l'index. Nous traiterons ici de la variante dite à *variables normalisées* : lors de la linéarisation d'un terme, chaque variable est remplacée par un symbole générique de la forme $*_i$, où i reflète l'ordre d'apparition de la variable dans le parcours préfixe. Ainsi, un terme $f(X,Y)$ devient $f - *_1 - *_2$ et un terme $f(X,X)$ devient $f - *_1 - *_1$. Ce choix facilite le filtrage, i.e. la sélection rapide des termes potentiellement unifiables avec un terme requête, lors de l'unification : la structure de l'index ne dépend plus des noms des variables, mais uniquement de leur *pattern* d'occurrence.

Lorsqu'on travaille avec des prédicats, on ajoute en tête de la *p-string* un symbole indiquant le *signe* du littéral : $+$ pour un prédicat positif $P(\ldots)$ et $\neg$ pour sa négation $\neg P(\ldots)$. Ainsi $P(f(X,Y))$ devient $+-P-f-*_1-*_2$ et $\neg P(f(X,Y))$ devient $\neg-P-f-*_1-*_2$. Cette distinction est essentielle pour la résolution : on ne cherche à unifier un littéral qu'avec des littéraux de signe opposé.

Exemple.

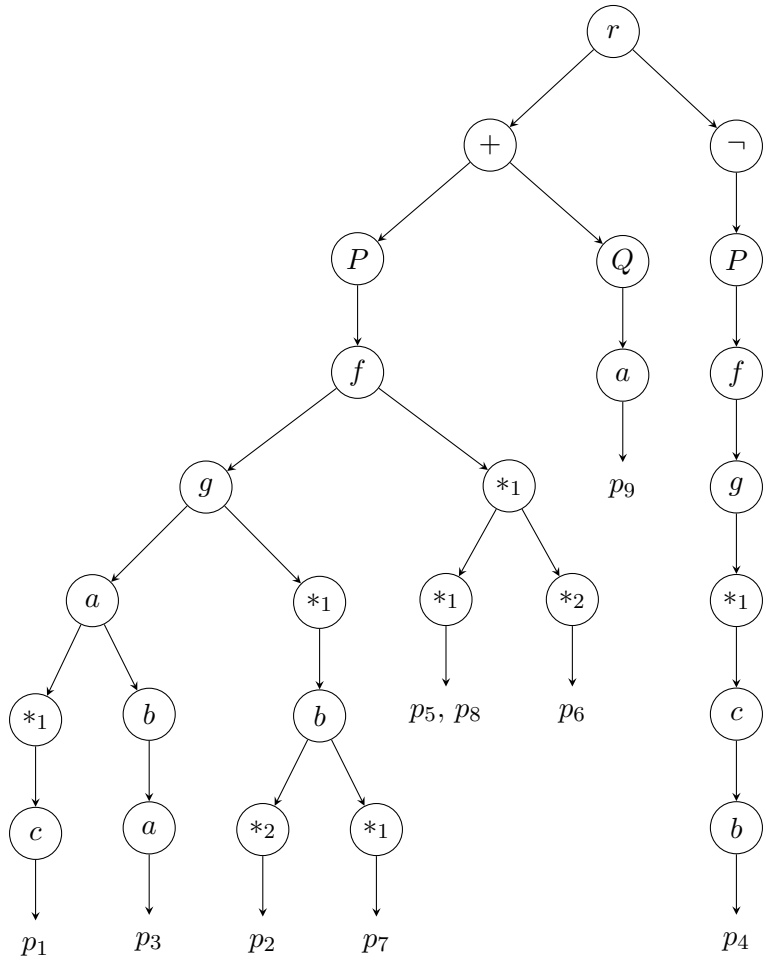
Prenons la base B définie comme suit :

$$B = \{ \underbrace{P(f(g(a,X),c))}_{p_1}, \quad \underbrace{P(f(g(X,b),Y))}_{p_2}, \quad \underbrace{P(f(g(a,b),a))}_{p_3}, \quad \underbrace{\neg P(f(g(X,c),b))}_{p_4},$$
$$\underbrace{P(f(X,X))}_{p_5}, \quad \underbrace{P(f(X,Y))}_{p_6}, \quad \underbrace{P(f(g(X,b),X))}_{p_7}, \quad \underbrace{P(f(Y,Y))}_{p_8}, \quad \underbrace{Q(a)}_{p_9} \}$$

On transforme ces prédicats en *p-strings normalisées* :

$$\begin{array}{lcl}
p_1 & \longrightarrow & + - P - f - g - a - *_1 - c \\
p_2 & \longrightarrow & + - P - f - g - *_1 - b - *_2 \\
p_3 & \longrightarrow & + - P - f - g - a - b - a \\
p_4 & \longrightarrow & \neg - P - f - g - *_1 - c - b \\
p_5 & \longrightarrow & + - P - f - *_1 - *_1 \\
p_6 & \longrightarrow & + - P - f - *_1 - *_2 \\
p_7 & \longrightarrow & + - P - f - g - *_1 - b - *_1 \\
p_8 & \longrightarrow & + - P - f - *_1 - *_1 \\
p_9 & \longrightarrow & + - Q - a
\end{array}$$

L'exemple précédent nous donne donc le *discrimination tree* suivant :



Avec cet exemple, on perçoit déjà l'utilité de la normalisation puisque les *p-strings normalisées* de p_5 et p_8 sont les mêmes. Ainsi l'arbre final ne produit qu'un chemin pour les eux prédicats.

> **Remarque.** *Cet exemple est tiré de* Term Indexing *- R. Sekar, I.V. Ramakrishnan et A. Voronkov[4]. Il a par ailleurs été adapté à notre version* normalisée *du discrimination tree.*

2.4.3. L'algorithme d'unification

L'unification par *discrimination tree* se déroule en deux phases distinctes. La première phase, dite de *filtrage*, consiste à parcourir l'index à partir d'un prédicat requête q, lui-même linéarisé en *p-string* par parcours préfixe. Les variables de q sont normalisées différemment de celles de l'index : on utilise la notation $?_i$ (contre $*_i$ pour les prédicats indexés), où i reflète l'ordre d'apparition dans le parcours préfixe. Cette distinction, entre les variables et d'autres symboles, est essentielle : lors du filtrage, une variable $?_i$ du terme requête est compatible avec *tout* nœud de l'arbre, symbole concret ou variable $*_j$, tandis qu'un symbole concret de q ne peut correspondre qu'à un nœud identique. À l'issue de ce parcours, on obtient un ensemble de *candidats*, i.e. les prédicats de la base dont la *p-string* est compatible avec celle de q. Il est a noté que pour notre version adaptée à la résolution de prédicats, si la *p-string* de q commence par $\neg$ (respectivement $+$) on filtrera les *p-strings* en cherchant le signe opposé : $+$ (respectivement $\neg$).

La seconde phase, dite d'*unification*, applique alors un algorithme d'unification plus classique, e.g. Robinson, entre q et chacun des candidats retenus, confirmant ou infirmant l'existence d'un unificateur. L'intérêt de cette approche réside dans le fait que le filtrage élimine rapidement et à moindre coût les termes non-unifiables, réduisant considérablement le nombre d'unifications effectives à réaliser. Cette phase est essentielle car le filtrage ne correspond qu'à la vérification d'une correspondance *locale* entre des symboles, il ne garantit pas la cohérence globale des substitutions. Cette contrainte est imposée par l'unification. À chaque étape du parcours, lorsqu'une variable $*_i$ est associée à un terme, cette association est mémorisée dans la substitution courante. Si la même variable $*_i$ réapparaît plus loin dans la branche, il faut vérifier que la nouvelle correspondace est compatible avec la substitutio déjà en mémoire, sinon la branche est abandonnée. Ainsi, la substitution n'est pas recalculée à chaque noeud : elle est conservée et propagée tout au long de l'exploration d'une branche afin de garantir la cohérence de l'unification sur l'ensemble du terme ou prédicat.

Par exemple, pour la requête

$$f(X, X)$$

et le terme stocké

$$f(a, b),$$

un filtrage purement local accepterait les correspondances

$$X \mapsto a \quad \text{et} \quad X \mapsto b.$$

Cependant, l'unification rejette ce cas, car une même variable ne peut pas être associée simultanément à deux valeurs différentes. C'est précisément pour détecter ce type de conflit que la substitution est conservée pendant tout le parcours de la branche.

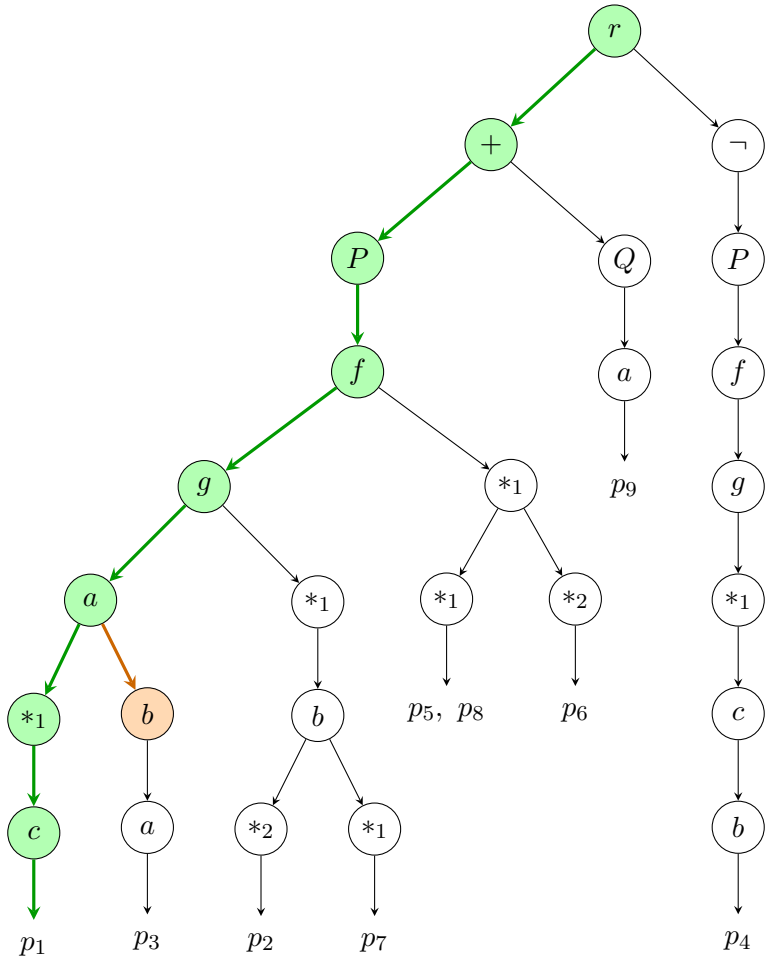
Exemple.

Considérons le prédicat requête $q = \neg P(f(g(a, Z), c))$, linéarisé et normalisé en $\neg - P - f - g - a - ?_1 - c$ et la même base B que dans l'exemple précédent.

Phase 1 : Filtrage. On descend dans le *discrimination tree* en suivant la *p-string* $\neg - P - f - g - a - ?_1 - c$:

- $\neg$: on suit la branche $+$, symbole inverse ;
- P : on suit la branche P, symbole identique ;
- f : on suit la branche f, symbole identique ;
- g : on suit la branche g, symbole identique ;
- a : on suit la branche a, symbole identique ;
- $?_1$: variable du requête, compatible avec tout nœud, on suit les branches $*_1$ (✓) et b (✓) ;
- c : depuis $*_1$, on suit c ✓ ; depuis b, aucune branche $c \Rightarrow$ éliminé.

Le filtrage retient uniquement $p_1 = P(f(g(a, X), c))$ comme candidat.



Phase 2 : Unification. On applique l'algorithme d'unification entre $q = \neg P(f(g(a, Z), c))$ et $p_1 = P(f(g(a, X), c))$:

$$\sigma = \{Z \mapsto X\} \quad \text{(ou symétriquement } \{X \mapsto Z\})$$

L'unification réussit avec la substitution σ. Les prédicats $p_2, \ldots, p_9$ n'ont jamais été considérés, illustrant l'efficacité du filtrage.

2.4.4. Complexité

L'analyse de la complexité des opérations sur un *discrimination tree* nécessite de distinguer deux types d'opérations principales : l'ajout d'un prédicat dans la structure et la recherche d'unification. Cette dernière est elle même décomposée en deux phases : le filtrage structurel puis l'unification effective.

Insertion.

L'insertion d'un prédicat dans un discrimination tree consiste à parcourir sa représentation linéarisée puis à insérer successivement chaque symbole dans l'arbre (ou le parcourir si le symbole est déjà partagé à une autre branche). L'algorithme suivant détaille ce processus :

Algorithm 4 Insertion d'un prédicat dans l'arbre de discrimination

```
Procédure InsererPredicat(Arbre, Predicat, Pointeur)
                                        ▷ Étape 1 : Linéarisation et normalisation des variables
    S ← MiseAPlat(Predicat)                                  ▷ Transforme le prédicat en p-string
                                        ▷ Étape 2 : Descente et construction de l'arbre
    NoeudCourant ← Arbre.racine
    Pour chaque Symbole dans S faire
        Si Symbole ∉ NoeudCourant.enfants alors
                                        ▷ Création d'un nouveau nœud si la branche n'existe pas
            NouveauNoeud ← CreerNoeud(Symbole)
            NoeudCourant.enfants[Symbole] ← NouveauNoeud
        Fin si
        NoeudCourant ← NoeudCourant.enfants[Symbole]
    Fin pour
                                        ▷ Étape 3 : Ajout du pointeur à la feuille
    Si Pointeur ∉ NoeudCourant.pointeurs alors
        NoeudCourant.pointeurs ← NoeudCourant.pointeurs ∪ {Pointeur}
    Fin si
Fin procédure
```

Soit n la taille d'un prédicat, i.e. son nombre total de symboles (par exemple, pour un prédicat $\neg P(X, g(X, b))$, $n = 6$ puisque sa *p-string* est $\neg - P - *_1 - g - *_1 - b$). Évaluons la complexité de chaque étape de l'algorithme :

- **Linéarisation (Étape 1) :** La fonction de mise à plat effectue un parcours en profondeur du prédicat. Elle visite chaque nœud (symbole, fonction ou variable) exactement une fois, ce qui requiert un temps proportionnel au nombre de symboles, soit $\mathcal{O}(n)$.
- **Construction (Étape 2) :** L'algorithme itère sur les n symboles de la *p-string*. Pour chaque symbole, la vérification de son existence parmi les enfants du nœud courant s'effectue en $\mathcal{O}(1)$ en moyenne, grâce à l'utilisation d'une table de hachage (*Dictionnaire* en Python). La création et l'ajout d'un nœud, si nécessaire, se font également en temps constant $\mathcal{O}(1)$. Cette boucle s'exécute donc en $\mathcal{O}(n)$.
- **Mise à jour (Étape 3) :** L'ajout du pointeur à la fin du chemin est une opération élémentaire en $\mathcal{O}(1)$.

La complexité temporelle globale de l'insertion d'un prédicat dans le *discrimination tree* est donc strictement linéaire par rapport à la taille du prédicat :

$$\mathcal{O}(n)$$

Du point de vue de la *complexité spatiale*, dans le pire des cas (où le prédicat ne partage aucun préfixe avec les termes déjà indexés dans l'arbre), l'algorithme allouera exactement n nouveaux nœuds en mémoire. La complexité spatiale par insertion est donc, elle aussi, au pire de $\mathcal{O}(n)$. Le partage de préfixes rend cependant cette empreinte mémoire bien plus faible en pratique.

Filtrage structurel.

Le filtrage structurel est la première phase de la recherche. Il consiste à parcourir l'arbre en suivant la *p-string* du terme requête. Contrairement à l'insertion, ce processus doit gérer le non-déterminisme induit par les variables présentes dans l'arbre (notées $*_i$) qui peuvent correspondre à n'importe quel sous-terme de la requête.

Algorithm 5 Filtrage structurel (Collecte de candidats)

```
Procédure CollecterCandidats(Noeud, Seq, i, Candidats)
    Si i ≥ Taille(Seq) alors                                  ▷ Condition d'arrêt : fin de la séquence
        Candidats ← Candidats ∪ Noeud.pointeurs
        Retourner
    Fin si
    s ← Seq[i]
                                          ▷ Cas 0 : Signe du prédicat (recherche de l'opposé pour résolution)
    Si s ∈ {'+', '¬'} alors
        oppose ← (s == '+'?'¬' : '+')
        Si oppose ∈ Noeud.enfants alors
            CollecterCandidats(Noeud.enfants[oppose], Seq, i + 1, Candidats)
        Fin si
        Retourner
    Fin si
                                                   ▷ Cas 1 : Symbole identique (hors variables)
    Si s ∈ Noeud.enfants et s n'est pas une variable alors
        CollecterCandidats(Noeud.enfants[s], Seq, i + 1, Candidats)
    Fin si
                                                 ▷ Cas 2 : Variable dans la requête (symbole '?')
    Si s est une variable requête alors
        Pour chaque Enfant dans Noeud.enfants faire
            Pour chaque DernierNoeud de SousTermes(Enfant, 1) faire
                CollecterCandidats(DernierNoeud, Seq, i + 1, Candidats)
            Fin pour
        Fin pour
        Retourner
    Fin si
                                                 ▷ Cas 3 : Variable dans l'arbre (symbole '*')
    prof ← CalculerProfondeur(i)
    Pour chaque (symb, enfant) dans Noeud.enfants faire
        Si symb est une variable arbre et i + prof ≤ Taille(Seq) alors
            CollecterCandidats(enfant, Seq, i + prof, Candidats)
        Fin si
    Fin pour
Fin procédure
```

Analyse de complexité : Soit n la taille du prédicat de requête et M le nombre total de nœuds dans l'arbre.

— **Pire cas :** Dans le cas d'une requête très générale ou d'un arbre fortement peuplé de variables, l'algorithme peut explorer plusieurs branches (backtracking). La complexité peut atteindre $\mathcal{O}(M)$ dans le pire des cas théorique.

— **Cas moyen :** Grâce à la structure d'index, on élimine massivement les branches incompatibles dès le début du parcours. La complexité moyenne est proche de $\mathcal{O}(n)$, rendant la recherche extrêmement performante par rapport à un parcours séquentiel de la base de données.

Unification effective.

Une fois le filtrage structurel terminé, nous obtenons un ensemble de candidats qui *ressemblent* à la requête. Cependant, l'arbre de discrimination « standard » ne capture pas les contraintes de co-occurrence des variables (par exemple, que le premier et le second argument doivent être identiques). Il est donc nécessaire de procéder à une unification classique sur les candidats restants.

Analyse de complexité : Soit k le nombre de candidats renvoyés par le filtrage ($k \ll$ taille de la base).

— L'opération dominante est l'appel à l'algorithme d'unification pour chaque candidat.

— Si l'unification est en $\mathcal{O}(n)$ (cas de Martelli-Montanari sans *occur-check* coûteux), la complexité de cette phase est $\mathcal{O}(k \cdot n)$.

Conclusion sur la performance : L'intérêt majeur du *Discrimination Tree* réside dans sa capacité à maintenir k très petit, souvent proche de 1, ce qui permet de passer d'une recherche en $\mathcal{O}(Base \cdot n)$ à une recherche quasi-linéaire $\mathcal{O}(n)$ par rapport à la taille de la requête.

3. Expérimentations

3.1. Organisation et implémentation

Organisation du travail.

La conduite de ce projet s'est inscrite dans une démarche collaborative inspirée du cadre *SCRUM*, une méthode agile privilégiant l'adaptabilité et la communication continue. *SCRUM* repose traditionnellement sur des cycles de travails courts appelés *sprints* et des points de synchronisation réguliers (*sprints review*). Dans notre cas, nous avons adopté un fonctionnement simplifié mais rigoureux, rythmé par des réunions hebdomadaires avec nos encadrants. Ces échanges nous ont permis de valider nos avancées techniques et de réévaluer nos priorités à chaque étape, garantissant ainsi une convergence constante vers les objectifs du projet.

Sur le plan technique, nous avons utilisé *Git*, permettant un développement asynchrone mais coordonné. Le choix du langage *Python* s'est imposé naturellement par notre expertise commune. Notre méthodologie de développement s'est articulée autour d'une progression logique :

— **Socle commun et standardisation :** Nous avons conçu collectivement la structure de données fondamentale pour la représentation des termes. Ce travail à trois était crucial pour établir une base stable avant d'entamer la phase de spécialisation. L'importance de ce socle réside dans sa *modularité* : en définissant une interface de programmation commune pour la manipulation des termes, nous avons pu garantir que chaque algorithme, bien qu'implémenté indépendamment, soit capable de traiter des données issues du même réservoir. Cette standardisation a non seulement facilité l'intégration finale, mais a surtout permis de réaliser des tests croisés rigoureux, où chaque algorithme d'unification pouvait être indifféremment sollicité sans nécessiter de conversion de format coûteuse ou risquée.

— **Développement distribué :** Chaque membre du groupe a ensuite pris la responsabilité de l'implémentation d'un algorithme spécifique : l'unification classique de *Robinson*, l'approche par système d'équations de *Martelli-Montanari*, ou l'indexation par *Discrimination Tree*.

— **Adaptation aux prédicats :** Une fois les cœurs algorithmiques stabilisés, nous avons collaboré pour étendre ces outils à la gestion des littéraux (prédicats signés), permettant le traitement de bases de connaissances réelles.

Détails d'implémentation et spécificités Python.

L'implémentation a tiré profit de la flexibilité de *Python*, notamment via l'usage intensif de *dictionnaires* pour le *discrimination tree* et d'*ensembles* pour *Martelli-Montanari*. Deux points d'ingénierie méritent d'être soulignés :

— **Modularité par encapsulation :** Afin de permettre une flexibilité totale dans le traitement des données, nous avons développé une structure « enveloppe » englobant les différents types de collections. Cette abstraction permet aux algorithmes de *Robinson* et de *Martelli-Montanari* de travailler de manière totalement agnostique vis-à-vis du stockage physique des prédicats. L'algorithme manipule une interface globale sans avoir à déterminer, en amont, s'il traite des *listes*, des *dictionnaires* ou des *ensembles* de prédicats. Cette déconnexion entre la logique de résolution et la représentation interne des collections rend le système évolutif et robuste.

— **Générateur de termes aléatoires :** Pour valider la robustesse de nos implémentations et préparer le *benchmark*, nous avons conçu un générateur automatique de termes. Cet outil permet de produire des structures de *profondeur* et de *complexité* variables, assurant ainsi une couverture de test exhaustive au-delà des cas manuels classiques.

Toutefois, le passage de la théorie à la pratique en *Python* a révélé certaines « pythonneries [1] » qui ont nécessité une attention particulière, comme la gestion des *objets mutables*. Le passage par référence en Python peut engendrer des effets de bord lors de l'application de substitutions successives sur un terme ; l'usage stratégique de copies profondes (*deepcopy*) ou de mécanismes d'immutabilité a été essentiel pour garantir l'intégrité des données pendant les calculs.

Architecture du projet.

Voici l'arborescence de notre projet, illustrant la séparation entre les outils de stockage des termes (*Util*), les algorithmes (*Core*) et les scripts de test. (l'arbre en dessous ne correspond pas au git mais on va modifier pour professionnaliser et mieux structurer le git, je le modifierai une fois la refonte structurelle faite)

```
/
├── Core
│   ├── robinson.py
│   ├── martelli_montanari.py
│   └── arbre_de_discrimination.py
├── Util
│   └── TermStore
│       ├── terme.py
│       └── generateur.py
└── Benchs
```

3.2. Protocole de benchmarks

Remarques générales :

1. Les **algorithmes** à tester sont : Robinson, Martelli-Montanari et le Discrimination Tree.
2. Les **structures de données** à tester sont : Listes, Ensembles, Dictionnaires et le Discrimination Tree.
3. D'après les points 1 et 2, le Discrimination Tree est un cas **à part**.
4. D'après les points précédents, il faudra donc tester :
   - **Robinson** sur des **Listes**
   - **Robinson** sur des **Ensembles**
   - **Robinson** sur des **Dictionnaires**
   - **Martelli-Montanari** sur des **Listes**
   - **Martelli-Montanari** sur des **Ensembles**
   - **Martelli-Montanari** sur des **Dictionnaires**
   - **Discrimination Tree**
5. Ces tests doivent être effectués sur les mêmes **banques de données**.
6. La **génération de ces données** représente un enjeu important pour ce protocole.
7. Ces données devront être : **conséquentes** (en nombre), **représentatives** (en cas) et **variées**.
8. Les **résultats** de ces différents tests doivent être **identiques**.

1. Terme déposé par l'équipe de développement et ses encadrants. Désigne les spécificités, parfois surprenantes, propres au langage Python.

9. Pour vérifier facilement que ces différents résultats soient les mêmes, ils doivent avoir le **même format**.
10. L'utilisation de Python ne doit pas être un facteur dans ces tests.
11. Des mesures intéréssantes à envisager :
    - Le temps d'exécution ;
    - Le **nombre d'opérations** effectuées ;
    - L'**espace mémoire** utilisé ;
    - La **taille** des termes (en nombre de caractères) ;
    - La **profondeur** des termes ;
    - La **taille** des query termes (en nombre de caractères) ;
    - La **profondeur** des query termes.
12. Il serait intéressant de tester plusieurs unifications (donc plusieurs query termes).
13. Il faudra préciser pour chaque tests les **paramètres** utilisés (signature de notre banque de données)
14. On pourra pour chaque couple (algo, structure) tester :
    - **La première** unification trouvée ;
    - **Toutes** les unifications possibles ;
15. Les différents tests devront être lancés depuis **la même machine**, dans **les mêmes conditions**.

Format d'entrée

- Plusieurs échelles (10 000, 100 000 puis 1 000 000) ;
- Les différentes échelles testeront les mêmes signatures ;
- Au sein de chaque échelle, plusieurs banques avec paramètres différents :
  - Profondeur max différente (1, 5, 10, 20) ;
  - Arité fixé à 10 ;
- Une signature par banque de données :
  - Symboles de variables ;
  - Symboles de fonctions et arités ;
  - Symboles de prédicats et arités ;
  - Symboles de constantes
- Littéraux aléatoires (Vérifier la "qualité" de l'aléa).

Format de sortie

- Un format de sortie facilement exploitable (CSV) ;
- Fichier de sortie contient : Jeu de données, Algo utilisé, Structure de données, Toute Unification ou la première, Iteration, Temps de Pretraitement, Temps Total, la RAM utilisée, CPU, Nombre d'Unifications.

3.3. Hypothèses

Dans une démarche scientifique rigoureuse, il est essentiel de formuler des *hypothèses* avant d'observer les résultats expérimentaux. Cela permet de ne pas se contenter d'une lecture passive des mesures, mais de *confronter* les observations à une réflexion théorique préalable : distinguer les résultats *attendus* des résultats *surprenants*, identifier les cas limites, et orienter l'interprétation. Les hypothèses que nous formulons ici sont issues de notre analyse théorique des structures et algorithmes étudiés, elles seront systématiquement confrontées aux mesures expérimentales dans la section d'analyse.

Nous organisons ces hypothèses selon trois axes : les *algorithmes*, les *structures de données*, et la *nature des données d'entrée*.

Hypothèses sur les algorithmes.

Du point de vue des algorithmes, nous anticipons une bien meilleure efficacité du *discrimination tree* dans le cas où l'on cherche toutes les unifications possible avec un littéral requête. En effet, la phase de filtrage éliminie rapidement les candidats non-unifiables, réduisant considérablement le nombre d'appels à l'algorithme d'unification effectif.

En revanche, dans le cas où l'on ne cherche que la première unification possible, cet avantage devient moins évident : le coût de construction et de parcours de l'index peut s'avérer supérieur au gain apporté par le filtrage, notamment si le premier candidat unifiable se trouve en début de base. Dans ce cas, un parcours séquentiel simple pourrait s'avérer plus rapide.

Concernant les deux algorithmes plus classiques, i.e. *Robinson et Martelli-Montanari*, nous ancitipons une légère supériorité de *Martelli-Montanari* sur *Robinson*. Ce dernier constitue la base historique de l'unification, tandis que *Martelli-Montanari* apporte un cadre formel plus optimisé, notamment grâce à son système de transformation de règles qui évite certains recalculs inutiles.

Hypothèses sur les structures de données.

Du point vue des structures de données, la notion centrale est celle de *compromis construction/utilisation* : plus une structure est sophistiquée, plus sa construction est coûteuse, et plus il faut amortir ce coût sur un grand nombre d'unifications pour en tirer un bénéfice réel.

Ainsi, la *liste* et l'*ensemble*, bien que peu coûteux à construire, imposent un parcours exhaustif de la base à chaque recherche. Leur avantage réside dans leur simplicité : ils sont performants sur des bases de petite taille ou lorsque la base n'est utilisée qu'une seule fois.

Le dictionnaire introduit une première forme d'indexation qui réduit le nombre de candidats à tester. Cependant son coût de construction reste non négligeable : sur des bases de petite taille ou des bases non réutilisées, le temps investi dans l'indexation risque de na pas être amorti.

Ce constat est encore plus marqué pour le *discrimination tree* : sa construction est particulièrement coûteuse, et son avantage ne se manifeste pleinement que sur des grandes bases, lorsque la base est réutilisée pour de nombreuses requêtes, et que l'on cherche toutes les unifications possibles. Sur des bases de petite taille, des bases non réutilisées, ou lorsqu'on ne cherche qu'une seule unification, sa construction peut s'avérer contre productive.

Hypothèses sur la nature des données d'entrées.

La nature des données d'entrée joue un rôle déterminant dans l'efficacité relative des structures. On distingue deux profils de comportement :

La *liste* et l'*ensemble* sont robustes face à la nature des données : leur comportement reste linéaire quelle que soit la structure des prédicats. La taille de la base est le seul facteur véritablement déterminant pour ces deux structures.

Le *dictionnaire* et le *discrimination tree* en revanche sont sensibles à la nature des données. Pour le *dictionnaire* une base présentant de nombreux symboles de prédicat différents favorise des groupes petits et donc une recherche efficace. À l'inverse, une base dominée par un seul symbole de prédicat fait dégénérer le *dictionnaire* en une simple *liste*.

Pour le *discrimination tree*, une base naturelle présentant de nombreux préfixes communs favorise la factorisation de l'arbre : celui-ci reste compact et le filtrage est très séléctif. À l'inverse, une base dont les prédicats partagent peu ou pas de préfixe communs réduit considérablement l'intéret de la factorisation, l'arbre dégénère alors et le filtrage perd en efficacité. De plus, une base dense en variables réduit aussi l'efficacité du filtrage : les variables étant compatibles avec tout noeud lors du parcours, peu de branches sont éliminées. À l'opposé, une base composée majoritairement de prédicats sans variables favorise un filtrage très séléctif, maximisant l'avantage du *discrimination tree*.

En résumé, *liste* et *ensemble* offrent des performances prévisibles et robustes au prix d'une efficacité limitée sur de grandes bases. Le *dictionnaire* et le *discrimination tree* sont potentiellement bien plus performants, mais leur efficacité dépend fortement de la nature des données et de la taille de la base.

3.4. Analyse des résultats

Conclusion

Références

[1] Jean Lieber. *Une introduction à la(des) logique(s) à destination d'étudiants de licence informatique*. Cours de L3. Consulté le : 07/05/2026. 2024.

[2] John Alan Robinson. "A machine-oriented logic based on the resolution principle". In : *Journal of the ACM (JACM)* 12.1 (1965), p. 23-41.

[3] Alberto Martelli et Ugo Montanari. "An efficient unification algorithm". In : *ACM Transactions on Programming Languages and Systems (TOPLAS)* 4.2 (1982), p. 258-282.

[4] R. Sekar, I. V. Ramakrishnan et Andrei Voronkov. "Term indexing". In : *Handbook of Automated Reasoning*. NLD : Elsevier Science Publishers B. V., 2001, p. 1853-1964. isbn : 0444508120.

[5] Hans de Nivelle. "An Algorithm for the Retrieval of Unifiers from Discrimination Trees". In : *Journal of Automated Reasoning* 20.1 (1998), p. 5-25. doi : `10.1023/A:1005992522805`. url : `https://doi.org/10.1023/A:1005992522805`.

[6] Robert Nieuwenhuis et al. "On the Evaluation of Indexing Techniques for Theorem Proving". In : *Automated Reasoning*. Sous la dir. de Rajeev Goré, Alexander Leitsch et Tobias Nipkow. Berlin, Heidelberg : Springer Berlin Heidelberg, 2001, p. 257-271. isbn : 978-3-540-45744-2.